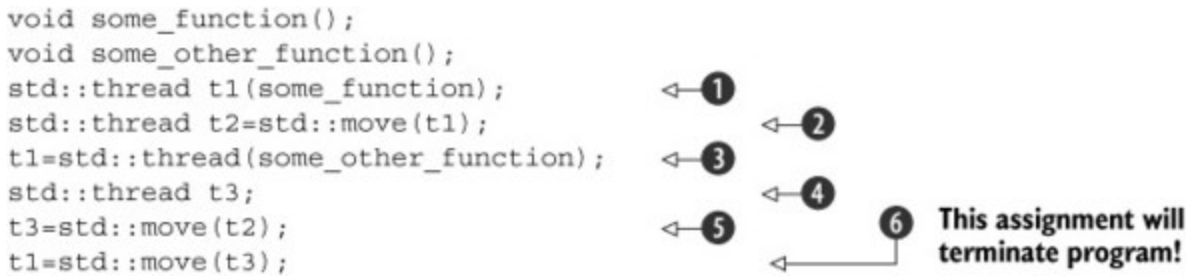
This is where the move support of `std::thread` comes in. As described in the previous section, many resource-owning types in the C++ Standard Library such as `std::ifstream` and `std::unique_ptr` are *movable* but not *copyable*, and `std::thread` is one of them. This means that the ownership of a particular thread of execution can be moved between `std::thread` instances, as in the following example. The example shows the creation of two threads of execution and the transfer of ownership of those threads among three `std::thread` instances, `t1`, `t2`, and `t3`:



First, a new thread is started ❶ and associated with `t1`. Ownership is then transferred over to `t2` when `t2` is constructed, by invoking `std::move()` to explicitly move ownership ❷. At this point, `t1` no longer has an associated thread of execution; the thread running `some_function` is now associated with `t2`.

Then, a new thread is started and associated with a temporary `std::thread` object ❸. The subsequent transfer of ownership into `t1` doesn't require a call to `std::move()` to explicitly move ownership, because the owner is a temporary object—moving from temporaries is automatic and implicit.

`t3` is default constructed ❹, which means that it's created without any associated thread of execution. Ownership of the thread currently associated with `t2` is transferred into `t3` ❺, again with an explicit call to `std::move()`, because `t2` is a named object. After all these moves, `t1` is associated with the thread running `some_other_function`, `t2` has no associated thread, and `t3` is associated with the thread running `some_function`.